

D1 — Threat Model

Bóveda Digital Segura de Documentos (SDDV) — UNAM 2026-2

Equipo SDDV: Barrios Aguilar · Contreras Colmenero · Martínez López · Caballero Martínez

Contents

D1 — Modelo de Amenazas (Threat Model) del SDDV	1
1. Alcance del sistema	1
2. Activos protegidos	1
3. Adversarios	2
4. Supuestos de confianza	2
5. Superficie de ataque (resumen)	3
6. Riesgos de la gestion de llaves (D6 — extension D1)	3
7. Sintesis: que protege D6 y que no	4
7. Requisitos de seguridad (RS) — referencia	5

D1 — Modelo de Amenazas (Threat Model) del SDDV

Este documento consolida el modelo de amenazas que hasta ahora vivia distribuido entre README.md, docs/D2_Encryption_Design.md y docs/D5_Signature_Design.md. Se mantiene aqui como referencia unica para las entregas D1–D6.

Estado: documento vivo. Se extiende en D6 con los riesgos especificos de la gestion de llaves (seccion 6).

1. Alcance del sistema

El SDDV es una “bodega digital segura” para documentos. El sistema:

- Cifra archivos (D2 — AEAD simetrico).
- Comparte archivos cifrados con multiples destinatarios (D3 — cifrado hibrido X25519 + KEM/DEM).
- Autentica el origen de cada archivo (D5 — firma Ed25519).
- Persiste las llaves privadas de cada identidad cifradas con un password (D6 — key management).

Fuera del alcance:

- Distribucion / publicacion de llaves publicas (PKI, web of trust).
- Proteccion contra ataques al sistema en ejecucion (malware, keyloggers, dumps de RAM).
- Almacenamiento confiable (se asume hostil; ver ADV-1).

2. Activos protegidos

Activo	Sensibilidad	Donde vive	Protegido por
Contenido de archivos	Alta	Contenedor SDDV/SDDH	AES-256-GCM (D2/D3)

Activo	Sensibilidad	Donde vive	Protegido por
Metadatos (filename, timestamp, recipients)	Media	Cabecera AAD del contenedor	Tag AEAD (D2/D3)
Llave privada Ed25519 (firma)	Critica	Archivo PEM PKCS8 / keystore JSON	Cifrado con password (D6)
Llave privada X25519 (cifrado)	Critica	(a partir de D6) keystore JSON	Cifrado con password (D6)
Password del usuario	Critica	Memoria del proceso, no en disco	UX (getpass), no caching (D6)
Firma digital del firmante	Media	Footer del contenedor firmado	Ed25519 (EUF-CMA)
Nonces AEAD	Media	Cabecera del contenedor	os.urandom + tag

3. Adversarios

ID	Adversario	Capacidades	Limitaciones (asunciones)
ADV-1	Atacante externo con acceso al almacenamiento	Lee, copia y modifica contenedores cifrados; copia el keystore	No puede romper AES-256 ni Ed25519; no observa el proceso en ejecucion
ADV-2	Destinatario malicioso	Lee su propio archivo legitimamente; intenta forjar contenedores que parezcan de otros	No tiene la privada del remitente
ADV-3	Man-in-the-Middle en la distribucion de claves publicas	Intercepta y sustituye llaves publicas en transito	No puede descifrar sin la privada correspondiente
ADV-4	Acceso fisico temporal al equipo	Copia el keystore completo (keystore/*.json)	No conoce el password; no instala malware persistente
ADV-5	Atacante que re-empaqueta firmas	Toma una firma valida y la reasocia a otro contenido o identidad	No puede recomputar la firma sin la privada
ADV-6 (fuera de scope)	Atacante con codigo ejecutandose en la maquina (malware)	RAM dump, keylogger, hooking de la libreria	— (fuera de scope explicito)

4. Supuestos de confianza

1. Los usuarios eligen contraseñas fuertes (≥ 12 caracteres). El sistema valida la fortaleza (`validate_password_strength`).
2. Las claves públicas distribuidas son auténticas (canal de distribución fuera de scope; D6 muestra el fingerprint para verificación out-of-band).
3. El SO provee CSPRNG seguro: `/dev/urandom` (Linux/macOS), `BCryptGenRandom` (Windows).
4. El almacenamiento es no confiable: contenedores y keystore se asumen accesibles para ADV-1 y ADV-4.
5. La aplicación no ha sido modificada (integridad del código fuera de scope; se asume entrega controlada).
6. No hay malware activo durante el uso (ADV-6 fuera de scope).
7. La librería `cryptography` de Python es correcta.

5. Superficie de ataque (resumen)

Punto de entrada	Riesgo	Requisito afectado	Mitigacion
Generacion de nonce	Reutilizacion catastrofica en GCM	RS-1, RS-2	<code>os.urandom(12)</code> + clave fresca por archivo
Keystore (D6)	Fuerza bruta offline sobre password	RS-4	<code>script cost</code> + <code>MIN_PASSWORD_LENGTH=12</code>
Importacion de claves publicas	MitM en distribucion	RS-3	Fuera de scope; fingerprints visibles
Entrada de filename	DoS / path traversal	RS-1	<code>validate_filename</code> + <code>safe_path_join</code> (VULN-001/006)
Entrada de timestamp	Replay de contenedores viejos	—	<code>validate_timestamp</code> (VULN-002)
Entrada de longitudes de campo	DoS por allocations	—	<code>validate_ciphertext_length</code> , <code>MAX_RECIPIENTS</code> (VULN-003/004)
Orden de verificacion	Descifrar antes de verificar	RS-3	<code>secure_verify_and_decrypt</code> lo impone

6. Riesgos de la gestion de llaves (D6 — extension D1)

Esta seccion analiza, escenario por escenario, que protege D6 y que no. El detalle del diseno criptografico (KDF, formato, ciclo de vida) esta en `docs/D6_Key_Management.md`.

6.1 Robo del keystore (ADV-1, ADV-4)

Capacidad del atacante. Obtiene una copia completa del directorio keystore/ (por ejemplo: lee un backup en la nube, copia el disco duro, accede al equipo desbloqueado durante 30 segundos).

Que protege D6. - La llave privada nunca esta en texto plano. Lo que ve el atacante es `encrypted_private_key` (base64 de AES-256-GCM ciphertext + tag) ademas del bloque `kdf` (salt + parametros) y `encryption` (nonce). - Para recuperar la privada el atacante debe ejecutar busqueda exhaustiva sobre el espacio de passwords. Cada intento exige: 1. `script(pwd, salt, n=215, r=8, p=1) → ~150 ms` y `~80 MiB RAM` en una laptop tipica. 2. `AES-256-GCM.decrypt(nonce, ciphertext, tag)` con la clave derivada. Si el password es incorrecto, AES-GCM lanza `InvalidTag` con probabilidad $\geq 1 - 2^{-128}$. - La memoria es el factor caro. Las GPUs y los ASIC se vuelven poco competitivos contra `script`: un AntMiner que rompe SHA-256 a 10^{14} hashes/s solo logra $\sim 10^5$ `script/s` con esos parametros.

Coste estimado de ataque, asumiendo: | Entropia del password | Intentos esperados (mitad del espacio) | Tiempo con 1 laptop | Tiempo con 1 000 GPUs eq. | |—|—|—|—| 40 bits ("Password2026!") | $2^{39} \approx 5.5 \times 10^{11}$ | ~ 2 600 anos | ~ 2.6 anos | | 50 bits (3 palabras Diceware) | $2^{49} \approx 5.6 \times 10^{14}$ | ~ 2.7 M anos | ~ 2 700 anos | | 65 bits (5 palabras Diceware) | $2^{64} \approx 1.8 \times 10^{19}$ | $\sim 8.8 \times 10^{10}$ anos | $\sim 10^8$ anos |

Conclusion: robar el keystore NO rompe la confidencialidad si el password tiene ≥ 50 bits de entropia. Con `MIN_PASSWORD_LENGTH=12` y composicion variada el caso tipico cae en la fila 2 o 3.

6.2 Password debil del usuario

Capacidad del atacante. Conoce o sospecha que el usuario eligio un password trivial (top-100k de listas filtradas, nombre del usuario + ano, etc.).

Que protege D6 parcialmente. - `validate_password_strength` rechaza: - vacios, - < 12 caracteres, - un solo caracter repetido. - El `script` eleva el costo por intento aun para passwords debiles: $100\,000$ candidatos $\times 150$ ms = 4 horas en una laptop, ~ 2 minutos con 100 GPUs equivalentes. Esto es lento, pero no infinito.

Lo que D6 NO hace (y por que): - No consulta listas de passwords filtrados (HaveIBeenPwned, etc.). Hacerlo requiere una API externa que cambia el modelo de privacidad (filtra el hash del password a un tercero). - No fuerza composicion (mayusculas + numeros + simbolos). La literatura reciente (NIST SP 800-63B) explicitamente desaconseja esas reglas: producen passwords MENOS aleatorios y mas predecibles. - La unica defensa robusta es educar al usuario a usar passphrases largas (Diceware: 5 palabras de un diccionario de 7776 dan ~65 bits — mas que suficiente).

6.3 Dispositivo comprometido (ADV-6)

Capacidad del atacante.Codigo malicioso ejecutandose con los privilegios del usuario victima. Ejemplos: malware en el navegador, troyano descargado, extension maliciosa, atacante con acceso fisico al equipo desbloqueado.

Que NO protege D6. - Un keylogger captura el password mientras se teclea (incluso si se usa getpass). - Un proceso con ptrace o SeDebugPrivilege en Windows lee la memoria del proceso SDDV en cualquier instante, incluyendo el momento del unlock cuando la privada esta en claro. - Un coldboot attack recupera la memoria RAM en segundos despues de apagar el equipo. - Un side-channel attack (timing, cache, power analysis) puede filtrar bits de la clave derivada durante scrypt.

Asuncion explicita del modelo de amenazas. D6 no defiende contra ADV-6. Esto es una limitacion estructural de cualquier sistema que maneje secretos en software sobre un SO compartido.

Mitigaciones fuera del scope del proyecto (mencionadas como roadmap): - HSM / YubiKey: la privada vive en hardware tamper-resistant; las operaciones (firma, descifrado de wrap-key) se delegan al token. - Zeroizacion de memoria con mlock + sobreescritura tras uso. - Aislamiento del proceso (SELinux, AppArmor, sandboxing).

6.4 Perdida del password

Escenario operacional. El usuario olvida el password operativo.

Mitigacion implementada. export_backup permite generar un .sddv_backup re-cifrado con un password de backup independiente. Si el usuario tiene el archivo y recuerda el password de backup, puede correr import_backup y restaurar la identidad (los fingerprints coinciden, los contenedores antiguos siguen siendo descifrables).

Si pierde ambos: la identidad es irrecuperable. Esta es una propiedad criptografica, no un bug que se pueda parchar. Una funcion de “reset” sin password constituye un bypass para un atacante (ADV-1 podria invocarla).

Recomendacion al usuario: generar el backup inmediatamente despues de crear la identidad, almacenarlo en un medio distinto del keystore activo (USB cifrado, gestor de contraseñas, papel en caja fuerte) y usar un password de backup que no se reutilice en ningun otro lugar.

7. Sintesis: que protege D6 y que no

Adversario / situacion	Cubierto por D6	Mitigacion
Lectura del keystore (ADV-1, ADV-4)	✓	scrypt + AES-256-GCM; coste offline disuasorio si pwd >= 50 bits
Password robusto en uso	✓	validate_password_strength >= 12 chars
Password debil tipo “123456”	parcial	scrypt frena (no infinito); educacion al usuario
Manipulacion del archivo del keystore	✓	AES-GCM tag invalida -> fail-closed
Rotacion de identidad comprometida	✓	rotate_keys + archivado

Adversario / situacion	Cubierto por D6	Mitigacion
Revocacion local	✓	revoke + IdentityRevokedError
Recuperacion tras olvido del pwd	✓	backup con pwd independiente
Distribucion autenticada de pub keys	x	fuera de scope; canal seguro asumido
Revocacion distribuida (CRL/OCSP)	x	fuera de scope
Dispositivo comprometido (ADV-6)	x	fuera de scope; requiere HSM
Perdida de pwd + perdida del backup	x	irrecuperable por diseno

7. Requisitos de seguridad (RS) — referencia

ID	Requisito	Cubierto por
RS-1	Confidencialidad 256-bit	AES-256-GCM (D2/D3)
RS-2	Integridad del contenido	Tag AEAD 128-bit + AAD (D2/D3)
RS-3	Autenticidad del remitente	Firma Ed25519 + verify-first (D5)
RS-4	Proteccion de llaves privadas	Keystore cifrado con KDF + AEAD (D6)
RS-5	Proteccion contra manipulacion	AAD + firma sobre todo el contenedor
RS-6	Unicidad de nonce	os.urandom por cifrado
RS-7	Gestion del ciclo de vida de llaves	KeyStore API: rotate/revoke/backup/restore (D6)